UNIVERSIDAD FRANZ TAMAYO

FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS

API REST DE PRODUCTOS CON FIREBASE

INTEGRANTES: BRAYAN QUISPE MAMANI

ASIGNATURA: PROGRAMACION DE DISPOSITIVOS MOVILES

DOCENTE:

M. Sc. MARCO ANTONIO SOTO V.

EL ALTO, BOLIVIA

2026

1. Objetivos y Fundamentación Teórica

1.1 Objetivo de la práctica

Desarrollar en Android Studio una aplicación móvil en Kotlin con Jetpack Compose que opere como cliente de una API REST de productos sobre Firebase Realtime Database, implementando las operaciones de listar, crear, editar y eliminar, y comprender cómo se comunica un dispositivo Android con una base de datos en la nube.

1.2 Fundamentación Teórica

Aplicaciones móviles Android: Android es un sistema operativo basado en el núcleo de Linux y adaptado para dispositivos de pantalla táctil. Sus aplicaciones se escriben principalmente en Kotlin o en Java, y se estructuran en componentes que el sistema va reciclando según el estado en el que se encuentra la aplicación.

Android Studio: Es el entorno de desarrollo oficial para crear aplicaciones Android. Integra editor de código, compilador, emulador y un inspector de interfaces, y administra las dependencias del proyecto mediante Gradle.

Lenguaje Kotlin: Kotlin es el lenguaje oficial recomendado por Google para Android desde 2019. Sobresale por su seguridad ante valores nulos, que se comprueban en tiempo de compilación, y por el uso de corrutinas, que permiten operaciones demoradas sin bloquear la interfaz.

Jetpack Compose: Es el kit de desarrollo de interfaces de Google que sustituye a los archivos de diseño XML. En lugar de describir la pantalla con etiquetas, el programador describe el estado y Compose se encarga de recomponer lo que debe redibujarse.

API REST: Es un estilo de arquitectura para servicios web que expone sus operaciones a través del protocolo HTTP, usando los métodos GET, POST, PUT, PATCH y DELETE sobre recursos identificados por una URL. Las respuestas se intercambian en formato JSON.

Firebase Realtime Database: Base de datos en la nube de tipo NoSQL que organiza la informacion como un arbol de claves y valores. Expone sus datos

mediante una API REST y, sobre todo, permite definir permisos de acceso con las denominadas Security Rules.

- GET: solicitar informacion
- POST: crear un recurso
- PATCH y PUT: modificar un recurso
- DELETE: eliminar un recurso

Estos elementos son la base del desarrollo realizado en la presente practica.

2. Herramientas y Recursos Utilizados

2.1 Software

- IDE: Android Studio 2026.1.2 (build 261.25134.95)
- Lenguaje de programación: Kotlin 2.2.10
- Gradle: 9.5.0, con Android Gradle Plugin 9.3.3
- SDK Android: minSdk 26, targetSdk 36, compilacion contra API 37
- Java Development Kit: Temurin 21

2.2 Hardware

- Computadora o laptop con Linux y JDK 21
- Dispositivo móvil Android: Samsung Galaxy A05M, Android 15 (API 35)
- Cable USB para la depuracion y la instalacion de la aplicacion

2.3 Recursos

- Consola de Firebase: creación del proyecto y de la base de datos
- Librerías de Android Jetpack usadas en la interfaz:
- Archivo local de respaldo en formato JSON
- Script de verificación en Bash para probar los cinco endpoints
- Documentación oficial de Firebase Realtime Database
    - Compose Material3
    - Compose Foundation
    - Lifecycle ViewModel

3. Metodología / Desarrollo Paso a Paso

3.1 Preparación del Entorno

- Creación del proyecto Firebase classfirebase-d856a desde la consola de Google.
- Creación de la base de datos Realtime Database en modo de prueba.
- Configuración de Android Studio con el SDK requerido y el JDK 21.
- Creación del proyecto Android con Kotlin y Jetpack Compose.
- Inclusión de las dependencias de Material3 y Foundation en build.gradle.kts.

3.2 Diseño de la Interfaz

- Division del codigo en cuatro capas: modelo, datos, controlador e interfaz.
- Pantalla principal con el título Productos y una lista de tarjetas.
- Cada tarjeta muestra nombre, descripción, precio en bolivianos y stock.
- Pantalla secundaria con un formulario de cuatro campos.
- Implementacion de los siguientes componentes:
    - Un boton flotante para registrar un producto nuevo
    - Botones Editar y Eliminar en cada tarjeta
    - Un cuadro de dialogo que pide confirmacion antes de borrar
    - Campos de texto con validacion de precio y stock

3.3 Programación de la Lógica y las Peticiones

- Construcción de las URL de lectura y escritura a partir de la direccion base.
- Ejecucion de las peticiones con HttpURLConnection.
- Asignación del método HTTP segun la operacion: GET, POST, PATCH y DELETE.
- Lectura de la respuesta JSON y conversion a objetos Producto.
- Ejecución del acceso a la red en un hilo secundario con Dispatchers.IO.
- Lectura de la direccion desde local.properties hacia BuildConfig con buildConfigField.

3.4 Manejo de Errores y Respaldo Local

- Traducción de los códigos de error HTTP a mensajes comprensibles:
- Mantenimiento de una copia de los datos en un archivo JSON interno.
- Escritura en el archivo local antes de intentar el envio a la red.
- Carga automatica de los datos al abrir la aplicacion, mediante LaunchedEffect.
    - 404: la base de datos aun no fue creada
    - 401 y 403: las reglas de seguridad bloquean el acceso

3.5 Pruebas y Validación

- Compilacion del proyecto sin errores ni advertencias.
- Ejecucion de las pruebas unitarias automaticas del modelo.
- Verificacion de los cinco endpoints con el script verificar-firebase.sh.
- Instalacion de la aplicacion en el dispositivo Samsung A05M por cable USB.
- Prueba manual del ciclo completo: listar, crear, editar y eliminar.

4. Resultados Obtenidos

- La aplicación logra listar, crear, editar y eliminar productos contra la base de datos de Firebase Realtime Database.
- Los cinco endpoints de la API respondieron con el codigo HTTP 200 en la verificacion.
- Cada operacion queda reflejada en la nube y tambien en el archivo JSON local.
- La aplicación se ejecuta correctamente en un dispositivo Samsung A05M con Android 15.
- Comprensión práctica de la comunicacion entre un dispositivo Android y un servicio REST.

5. Dificultades Encontradas

La mayor parte del tiempo se perdio en un problema que no estaba en el codigo. La aplicacion compilaba bien, no mostraba errores, y aun asi no guardaba nada en la nube. Resulto ser que el proyecto de Firebase no tenia creada ninguna base de datos, y todas las peticiones respondian con 404. Lo confuso es que el proyecto traia una clave secreta que al principio parecio la solucion, pero Firebase elimino las database secrets en 2024, de modo que nunca funciono. Tambien se perdio tiempo configurando el archivo local, que seguia con un marcador de posición en vez de la direccion real.

6. Conclusiones

Se logró completar la aplicación funcional de extremo a extremo, con las cuatro operaciones CRUD verificadas tanto desde la consola de Firebase como desde el telefono. Lo que mas se aprendio no fue a escribir el codigo, sino a diagnosticar: comprobar el comportamiento real del servidor en lugar de suponer que el problema estaba en la aplicacion. Tambien quedo claro que la seguridad en Firebase no depende de ocultar la direccion ni de guardar claves en el codigo, sino de configurar correctamente las reglas de seguridad.

7. Anexos



Figura 1. Pantalla principal con el listado de productos descargado de Firebase.

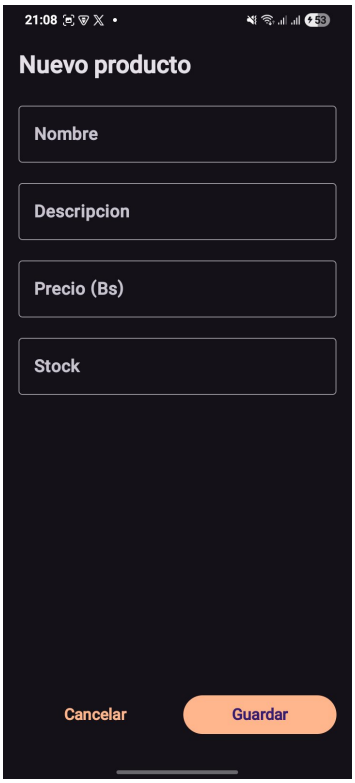


Figura 2. Formulario de registro con los campos de nombre, descripción, precio y stock.

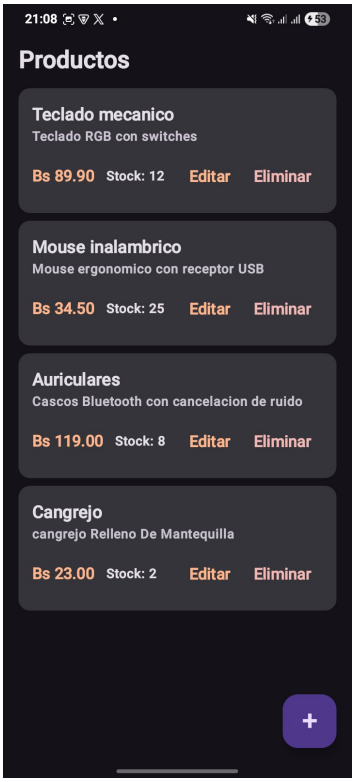


Figura 3. Cuadro de diálogo que solicita confirmación antes de eliminar un producto.

7.1 Evidencia de configuracion

Las siguientes figuras documentan la configuracionrealizada durante la practica: la base de datos en la consola de Firebase, las reglas de seguridad publicadas, y los fragmentos de codigo que hacen posible la conexion.

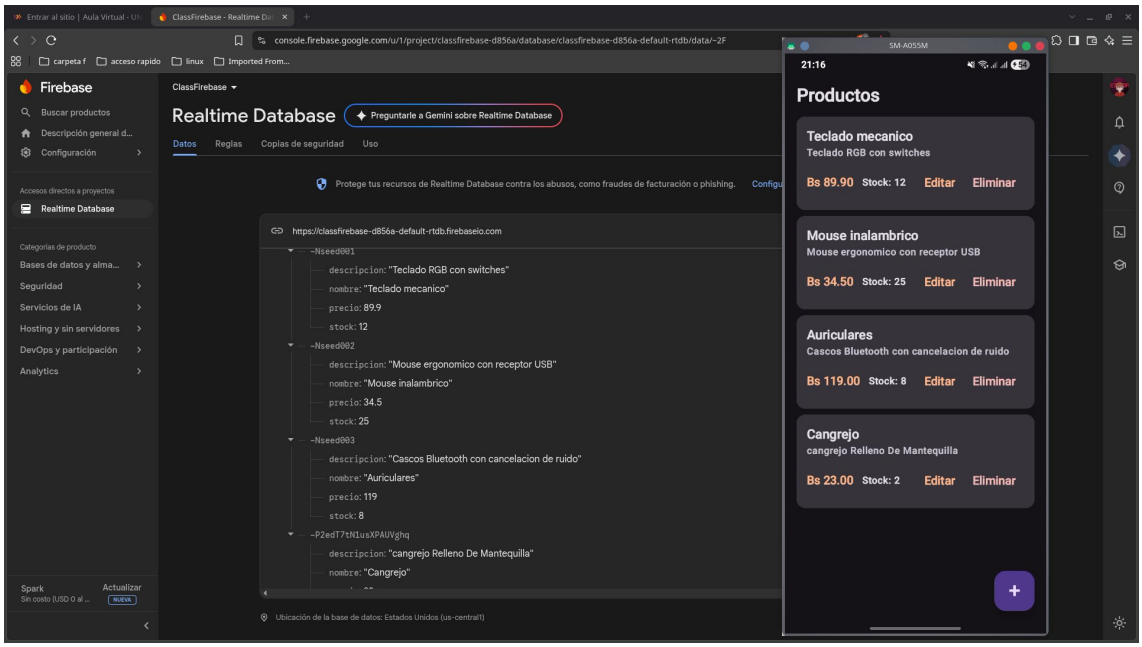


Figura 4. La app Android y la base de datos en Firebase mostrando los mismos productos.

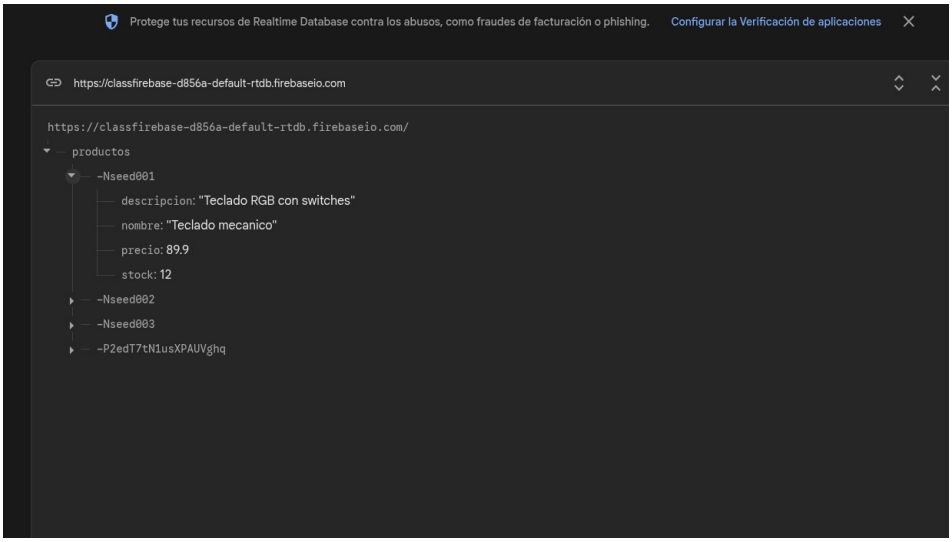


Figura 5. Consola de Firebase con la URL de la base Realtime Database y los productos almacenados.

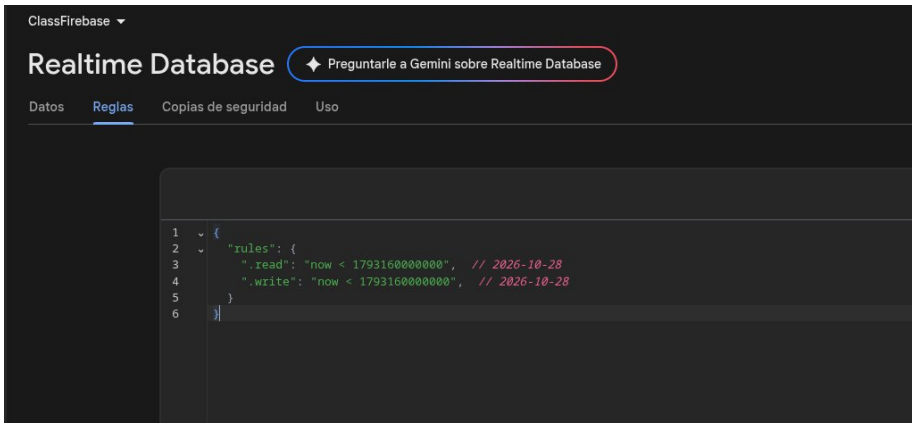


Figura 6. Reglas de seguridad publicadas: acceso de lectura y escritura hasta la fecha 28/10/2026.



Figura 7. Lectura de local.properties en el modulo Gradle.



Figura 8. Exposicion de la URL y el token como campos de BuildConfig.

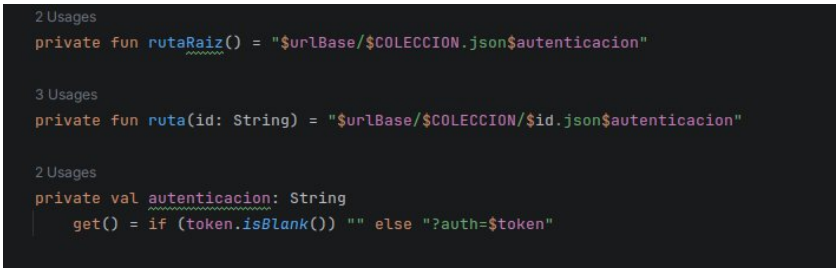
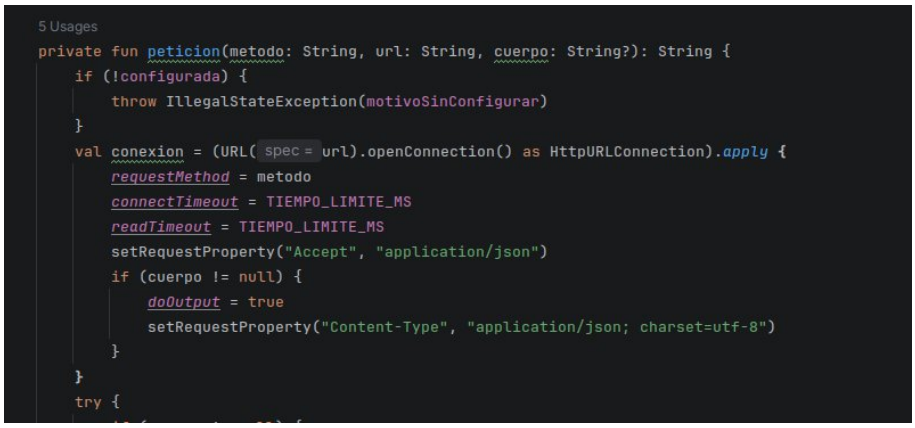
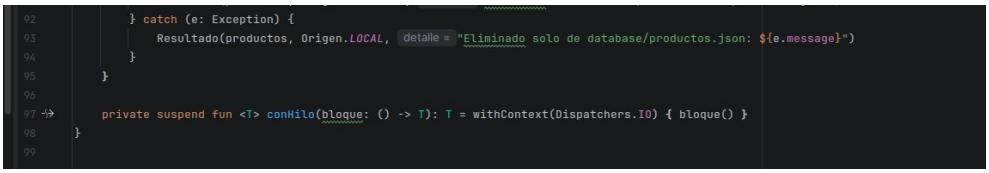


Figura 9. Construccion de las rutas REST, con el sufijo .json obligatorio.



```
5 Usages
private fun peticion(metodo: String, url: String, cuerpo: String?): String {
    if (!configurada) {
        throw IllegalStateException(motivoSinConfigurar)
    }
    val conexion = (URL( spec = url).openConnection() as HttpURLConnection).apply {
        requestMethod = metodo
        connectTimeout = TIEMPO_LIMITE_MS
        readTimeout = TIEMPO_LIMITE_MS
        setRequestProperty("Accept", "application/json")
        if (cuerpo != null) {
            doOutput = true
            setRequestProperty("Content-Type", "application/json; charset=utf-8")
        }
    }
    try {
```

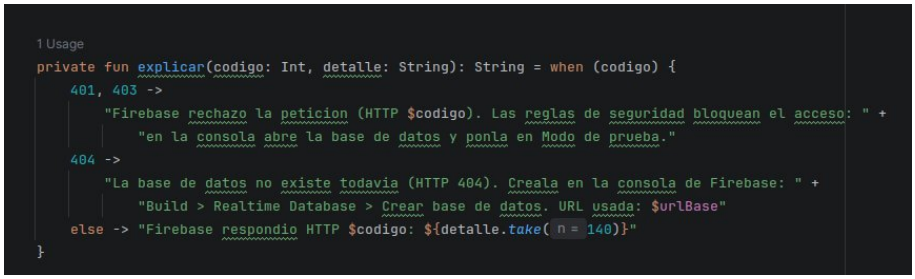
Figura 10. Metodo peticion(): apertura de la conexion HTTP, metodo y limites de espera.



```
        } catch (e: Exception) {
            Resultado(productos, Origen.LOCAL, detalle = "Eliminado solo de database/productos.json: ${e.message}")
        }
    }

    private suspend fun <T> conHilo(bloque: () -> T): T = withContext(Dispatchers.IO) { bloque() }
}
```

Figura 11. Envio de todo el acceso a datos a un hilo secundario con Dispatchers.IO.



```
1 Usage
private fun explicar(codigo: Int, detalle: String): String = when (codigo) {
    401, 403 ->
        "Firebase rechazo la peticion (HTTP $codigo). Las reglas de seguridad bloquean el acceso: " +
            "en la consola abre la base de datos y ponla en Modo de prueba."
    404 ->
        "La base de datos no existe todavia (HTTP 404). Creala en la consola de Firebase: " +
            "Build > Realtime Database > Crear base de datos. URL usada: $urlBase"
    else -> "Firebase respondio HTTP $codigo: ${detalle.take( n = 140)}"
}
```

Figura 12. Traduccion de los codigos de error de Firebase a mensajes utiles para el usuario.